

Introduction to Django REST Framework

Follow this website: [Django REST Framework](#)

In REST Framework we use Class based View

Installation of REST Framework

Setting up a new environment

Before we do anything else we'll create a new virtual environment, using **venv**. This will make sure our package configuration is kept nicely isolated from any other projects we're working on.

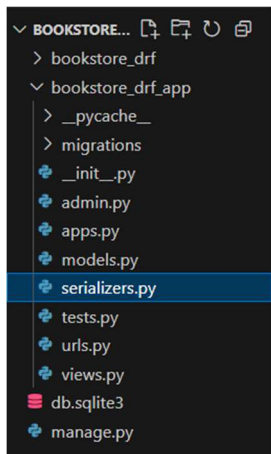
```
python3 -m venv env
source env/bin/activate
```

Now that we're inside a virtual environment, we can install our package requirements.

```
pip install django
pip install djangorestframework
pip install pygments # We'll be using this for the code highlighting
```

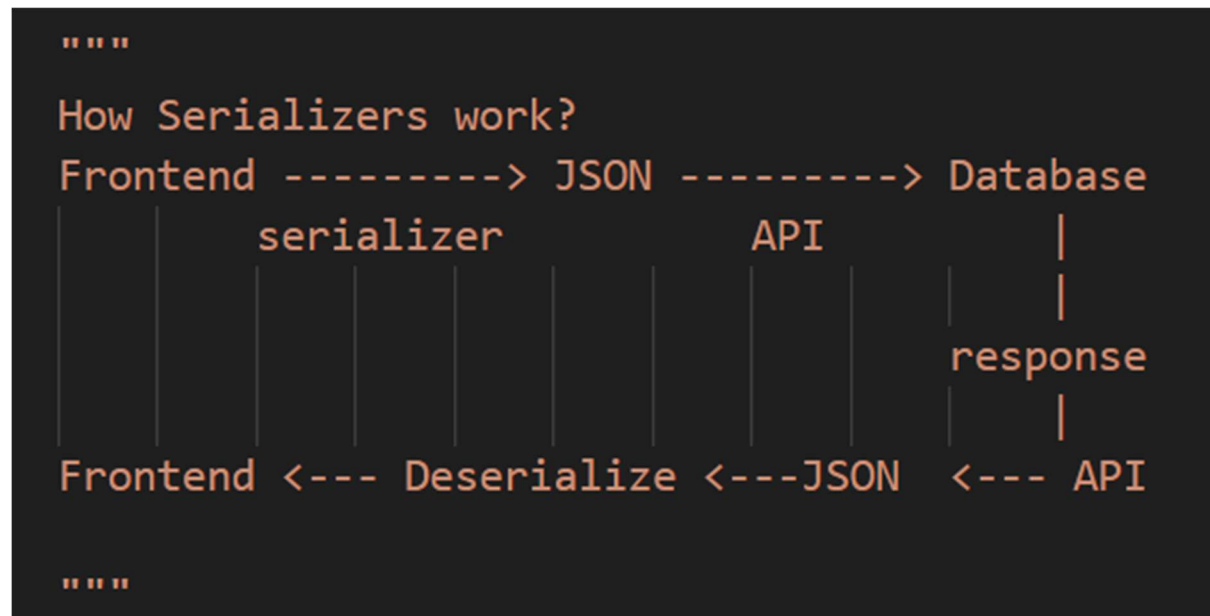
```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'snippets',
]
```

In app create a file 'serializers.py':

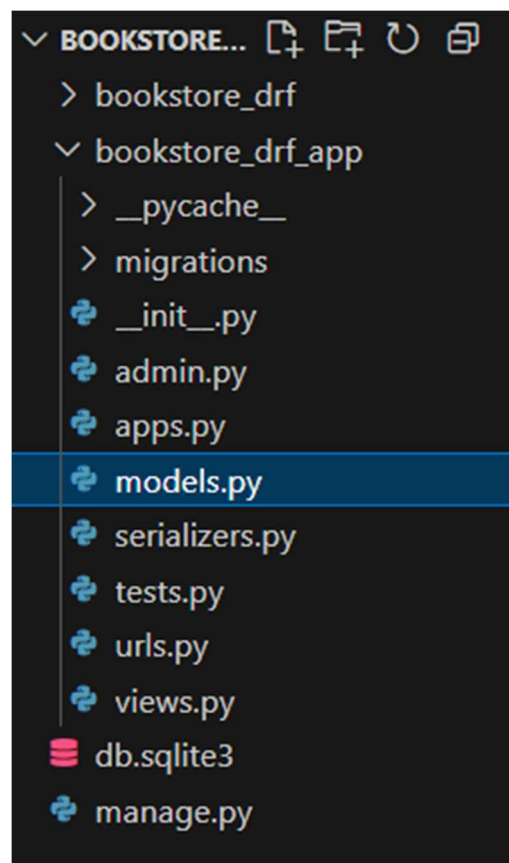


What serializers do?

To send the data from frontend to backend (transfers it into json format)



Then create a file “models.py”:



serializers.py

Using ModelSerializers

Our `SnippetSerializer` class is replicating a lot of information that's also contained in the `Snippet` model. It would be nice if we could keep our code a bit more concise.

In the same way that Django provides both `Form` classes and `ModelForm` classes, REST framework includes both `Serializer` classes, and `ModelSerializer` classes.

Let's look at refactoring our serializer using the `ModelSerializer` class. Open the file `snippets/serializers.py` again, and replace the `SnippetSerializer` class with the following.

```
class SnippetSerializer(serializers.ModelSerializer):
    class Meta:
        model = Snippet
        fields = ['id', 'title', 'code', 'linenos', 'language', 'style']
```

There are three types of views in this process:

- i) API View → Long Cut
- ii) Generic View → Medium Cut
- iii) View set and Router → Short Cut

For API View (views.py) => This is only for 'get' and 'post' request

Tutorial 3: Class-based Views

We can also write our API views using class-based views, rather than function based views. As we'll see this is a powerful pattern that allows us to reuse common functionality, and helps us keep our code **DRY**.

Rewriting our API using class-based views

We'll start by rewriting the root view as a class-based view. All this involves is a little bit of refactoring of `views.py`.

```
from snippets.models import Snippet
from snippets.serializers import SnippetSerializer
from django.http import Http404
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

class SnippetList(APIView):
    """
    List all snippets, or create a new snippet.
    """
    def get(self, request, format=None):
        snippets = Snippet.objects.all()
        serializer = SnippetSerializer(snippets, many=True)
        return Response(serializer.data)

    def post(self, request, format=None):
        serializer = SnippetSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

To handle CORS Policy:

Follow this Link: [CORS Policy](#)

First Install:

Steps to allow CORS in your Django Project –

1. Install django-cors-headers using PIP:

```
pip install django-cors-headers
```

```
(base) C:\Users\dell-pc>pip install django-cors-headers
Collecting django-cors-headers
  Downloading https://files.pythonhosted.org/packages/51/99/41898d5eddb68bc5545c08f5315e00d6036cc6f1aba5fd824042dd1b04ea
/django_cors_headers-3.5.0-py3-none-any.whl
Requirement already satisfied: Django>=2.2 in c:\users\dell-pc\anaconda3\lib\site-packages (from django-cors-headers) (2
.2.5)
Requirement already satisfied: sqlparse in c:\users\dell-pc\anaconda3\lib\site-packages (from Django>=2.2->django-cors-h
eaders) (0.2.4)
Requirement already satisfied: pytz in c:\users\dell-pc\anaconda3\lib\site-packages (from Django>=2.2->django-cors-heade
rs) (2018.7)
Installing collected packages: django-cors-headers
Successfully installed django-cors-headers-3.5.0
```

Then...

2. Add corsheaders to installed applications section in the settings.py file:

```
INSTALLED_APPS = [

    ...

    'corsheaders',

    ...

]
```

```
33
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41     'enum_auth',
42     'rest_framework',
43     'rest_framework.authtoken',
44     'corsheaders',
45 ]
46
```

After that in middleware...

3. Add corsheaders.middleware.CorsMiddleware to middleware section in settings.py file:

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    ...  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
    'corsheaders.middleware.CorsMiddleware',  
]
```

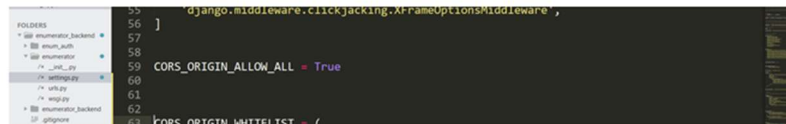


```
44     'corsheaders',  
45 ]  
46  
47 MIDDLEWARE = [  
48     'corsheaders.middleware.CorsMiddleware',  
49     'django.middleware.security.SecurityMiddleware',  
50     'django.contrib.sessions.middleware.SessionMiddleware',  
51     'django.middleware.common.CommonMiddleware',  
52     'django.middleware.csrf.CsrfViewMiddleware',
```

Allow the origin:

4. If you want to allow access for all domains, set the following variable to TRUE in settings.py file:

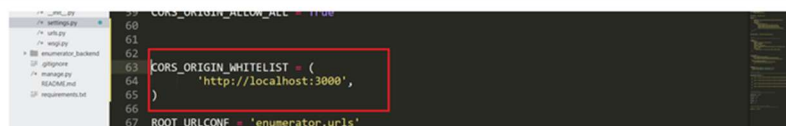
```
CORS_ORIGIN_ALLOW_ALL = True
```



```
59     'django.middleware.clickjacking.XFrameOptionsMiddleware',  
60 ]  
61  
62 CORS_ORIGIN_ALLOW_ALL = True  
63  
64 CORS_ORIGIN_WHITELIST = (  
65     'http://localhost:8000',  
66 )  
67  
68 ROOT_URLCONF = 'enumerator.urls'
```

Alternatively, you can specify which domains you want to give access to by doing the following in settings.py file:

```
CORS_ORIGIN_ALLOW_ALL = False  
CORS_ORIGIN_WHITELIST = (  
    'http://localhost:8000',  
)
```



```
60  
61  
62 CORS_ORIGIN_ALLOW_ALL = True  
63  
64 CORS_ORIGIN_WHITELIST = (  
65     'http://localhost:8000',  
66 )  
67  
68 ROOT_URLCONF = 'enumerator.urls'
```

That's all! Now your API is accessible to other applications hosted on other selected servers.

For API View (views.py)

=> This is only for 'get', 'update' and 'delete' request

So far, so good. It looks pretty similar to the previous case, but we've got better separation between the different HTTP methods. We'll also need to update the instance view in `views.py`.

```
class SnippetDetail(APIView):
    """
    Retrieve, update or delete a snippet instance.
    """
    def get_object(self, pk):
        try:
            return Snippet.objects.get(pk=pk)
        except Snippet.DoesNotExist:
            raise Http404

    def get(self, request, pk, format=None):
        snippet = self.get_object(pk)
        serializer = SnippetSerializer(snippet)
        return Response(serializer.data)

    def put(self, request, pk, format=None):
        snippet = self.get_object(pk)
        serializer = SnippetSerializer(snippet, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    def delete(self, request, pk, format=None):
        snippet = self.get_object(pk)
        snippet.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

For Generic class view

⇒ get, update, post, delete

Using generic class-based views

Using the mixin classes we've rewritten the views to use slightly less code than before, but we can go one step further. REST framework provides a set of already mixed-in generic views that we can use to trim down our `views.py` module even more.

```
from snippets.models import Snippet
from snippets.serializers import SnippetSerializer
from rest_framework import generics

class SnippetList(generics.ListCreateAPIView):
    queryset = Snippet.objects.all()
    serializer_class = SnippetSerializer

class SnippetDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Snippet.objects.all()
    serializer_class = SnippetSerializer
```

For Viewsets and Router

Tutorial 6: ViewSets & Routers

REST framework includes an abstraction for dealing with `ViewSet`, that allows the developer to concentrate on modeling the state and interactions of the API, and leave the URL construction to be handled automatically, based on common conventions.

`ViewSet` classes are almost the same thing as `View` classes, except that they provide operations such as `retrieve`, or `update`, and not method handlers such as `get` or `put`.

A `ViewSet` class is only bound to a set of method handlers at the last moment, when it is instantiated into a set of views, typically by using a `Router` class which handles the complexities of defining the URL conf for you.

Refactoring to use ViewSets

Let's take our current set of views, and refactor them into view sets.

First of all let's refactor our `UserList` and `UserDetail` classes into a single `UserViewSet` class. In the `snippets/views.py` file, we can remove the two view classes and replace them with a single ViewSet class:

```
from rest_framework import viewsets

class UserViewSet(viewsets.ReadOnlyModelViewSet):
    """
    This viewset automatically provides `list` and `retrieve` actions.
    """
    queryset = User.objects.all()
    serializer_class = UserSerializer
```

Using Routers

Because we're using `ViewSet` classes rather than `View` classes, we actually don't need to design the URL conf ourselves. The conventions for wiring up resources into views and urls can be handled automatically, using a `Router` class. All we need to do is register the appropriate view sets with a router, and let it do the rest.

Here's our re-wired `snippets/urls.py` file.

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter

from snippets import views

# Create a router and register our ViewSets with it.
router = DefaultRouter()
router.register(r'snippets', views.SnippetViewSet, basename='snippet')
router.register(r'users', views.UserViewSet, basename='user')

# The API URLs are now determined automatically by the router.
urlpatterns = [
    path('', include(router.urls)),
]
```

Working with Authentication and Authorization

Permissions

If we want no one but authenticated user can access url then:

Go to API Guide > Permissions > Setting the permission policy

In your code file (settings.py)

Setting the permission policy

The default permission policy may be set globally, using the `DEFAULT_PERMISSION_CLASSES` setting. For example.

```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ]
}
```

You can also do this for a particular view

You can also set the authentication policy on a per-view, or per-viewset basis, using the `APIView` class-based views.

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView

class ExampleView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, format=None):
        content = {
            'status': 'request was permitted'
        }
        return Response(content)
```

Custom Permissions

Follow this GitHub Link: [REST Framework GitHub, Custom Permissions](#)

```
first_app > permissions.py > ReviewerOrReadOnly > has_object_permission
1  from rest_framework import permissions
2  class AdminOrReadOnly(permissions.IsAdminUser):
3      def has_permission(self, request, view):
4          if request.method in permissions.SAFE_METHODS: # get
5              return True
6          else: # put, delete, post
7              bool(request.user and request.user.is_staff)
8
9  class ReviewerOrReadOnly(permissions.BasePermission):
10     def has_object_permission(self, request, view, obj):
11         if request.method in permissions.SAFE_METHODS:
12             return True
13         else:
14             return obj.user == request.user
15
```

Authentication

Go to API Guides > Authentication > Basic Authentication (test only) > Token Authentication

In settings.py →

TokenAuthentication

Note: The token authentication provided by Django REST framework is a fairly simple implementation.

For an implementation which allows more than one token per user, has some tighter security implementation details, and supports token expiry, please see the [Django REST Knox](#) third party package.

This authentication scheme uses a simple token-based HTTP Authentication scheme. Token authentication is appropriate for client-server setups, such as native desktop and mobile clients.

To use the `TokenAuthentication` scheme you'll need to **configure the authentication classes** to include `TokenAuthentication`, and additionally include `rest_framework.authtoken` in your `INSTALLED_APPS` setting:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework.authtoken'  
]
```

Make sure to run `manage.py migrate` after changing your settings.

Setting the authentication scheme

The default authentication schemes may be set globally, using the `DEFAULT_AUTHENTICATION_CLASSES` setting. For example.

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.BasicAuthentication',  
        'rest_framework.authentication.SessionAuthentication',  
    ]  
}
```

Generating Tokens

Generating Tokens

By using signals

If you want every user to have an automatically generated Token, you can simply catch the User's `post_save` signal.

```
from django.conf import settings
from django.db.models.signals import post_save
from django.dispatch import receiver
from rest_framework.authtoken.models import Token

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_auth_token(sender, instance=None, created=False, **kwargs):
    if created:
        Token.objects.create(user=instance)
```

More About DRF (Filtering, Ordering, Pagination...)

Filtering against query parameters

A final example of filtering the initial queryset would be to determine the initial queryset based on query parameters in the url.

We can override `.get_queryset()` to deal with URLs such as `http://example.com/api/purchases?username=denvercoder9`, and filter the queryset only if the `username` parameter is included in the URL:

```
class PurchaseList(generics.ListAPIView):
    serializer_class = PurchaseSerializer

    def get_queryset(self):
        """
        Optionally restricts the returned purchases to a given user,
        by filtering against a `username` query parameter in the URL.
        """
        queryset = Purchase.objects.all()
        username = self.request.query_params.get('username')
        if username is not None:
            queryset = queryset.filter(purchaser__username=username)
        return queryset
```

Filter Link: [Django Filtering](#), [Integration with DRF](#), [Filter*](#)

Installation

Django-filter can be installed from PyPI with tools like `pip`:

```
$ pip install django-filter
```

Then add `'django_filters'` to your `INSTALLED_APPS`.

```
INSTALLED_APPS = [  
    ...  
    'django_filters',  
]
```

You can add in 2 ways:

1st is by default filtering is same for everyone

You should now either add the filter backend to your settings:

```
REST_FRAMEWORK = {  
    'DEFAULT_FILTER_BACKENDS': ['django_filters.rest_framework.DjangoFilterBackend']  
}
```

2nd is custom filter as you wish

Or add the filter backend to an individual View or ViewSet.

```
from django_filters.rest_framework import DjangoFilterBackend  
  
class UserListView(generics.ListAPIView):  
    ...  
    filter_backends = [DjangoFilterBackend]
```

If all you need is simple equality-based filtering, you can set a `filterset_fields` attribute on the view, or viewset, listing the set of fields you wish to filter against.

```
class ProductList(generics.ListAPIView):  
    queryset = Product.objects.all()  
    serializer_class = ProductSerializer  
    filter_backends = [DjangoFilterBackend]  
    filterset_fields = ['category', 'in_stock']
```

Searching Filter

SearchFilter

The `SearchFilter` class supports simple single query parameter based searching, and is based on the [Django admin's search functionality](#).

When in use, the browsable API will include a `SearchFilter` control:

Search

The `SearchFilter` class will only be applied if the view has a `search_fields` attribute set. The `search_fields` attribute should be a list of names of text type fields on the model, such as `CharField` or `TextField`.

```
from rest_framework import filters

class UserListView(generics.ListAPIView):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ['username', 'email']
```

Ordering Filter

Specifying which fields may be ordered against

It's recommended that you explicitly specify which fields the API should allow in the ordering filter. You can do this by setting an `ordering_fields` attribute on the view, like so:

```
class UserListView(generics.ListAPIView):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    filter_backends = [filters.OrderingFilter]
    ordering_fields = ['username', 'email']
```

The client may also specify reverse orderings by prefixing the field name with '-', like so:

```
http://example.com/api/users?ordering=-username
```

You can read the documents for more clarification.

Pagination

Pagination Three Types:

PageNumberPagination

This pagination style accepts a single number page number in the request query parameters.

Request

```
GET https://api.example.org/accounts/?page=4
```

Setup

To enable the `PageNumberPagination` style globally, use the following configuration, and set the `PAGE_SIZE` as desired:

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 100
}
```

PageNumberPagination

This pagination style accepts a single number page number in the request query parameters.

Request

```
GET https://api.example.org/accounts/?page=4
```

Configuration

The `PageNumberPagination` class includes a number of attributes that may be overridden to modify the pagination style.

To set these attributes you should override the `PageNumberPagination` class, and then enable your custom pagination class as above.

- `django_paginator_class` - The Django Paginator class to use. Default is `django.core.paginator.Paginator`, which should be fine for most use cases.
- `page_size` - A numeric value indicating the page size. If set, this overrides the `PAGE_SIZE` setting. Defaults to the same value as the `PAGE_SIZE` settings key.
- `page_query_param` - A string value indicating the name of the query parameter to use for the pagination control.
- `page_size_query_param` - If set, this is a string value indicating the name of a query parameter that allows the client to set the page size on a per-request basis. Defaults to `None`, indicating that the client may not control the requested page size.
- `max_page_size` - If set, this is a numeric value indicating the maximum allowable requested page size. This attribute is only valid if `page_size_query_param` is also set.
- `last_page_strings` - A list or tuple of string values indicating values that may be used with the `page_query_param` to request the final page in the set. Defaults to `('last',)`.
- `template` - The name of a template to use when rendering pagination controls in the browsable API. May be overridden to modify the rendering style, or set to `None` to disable HTML pagination controls completely. Defaults to `"rest_framework/pagination/numbers.html"`.

```

first_app > paginations.py > ProductPagination
1 from rest_framework.pagination import PageNumberPagination
2
3 class ProductPagination(PageNumberPagination):
4     page_size = 2 # every page contains 2 elements
5     page_query_param = 'pg' # you can name the page such as pg
6
7     # you can modify the elements as how much elements you want to see per page
8     page_size_query_param = 'size'

```

LimitOffsetPagination

This pagination style mirrors the syntax used when looking up multiple database records. The client includes both a "limit" and an "offset" query parameter. The limit indicates the maximum number of items to return, and is equivalent to the `page_size` in other styles. The offset indicates the starting position of the query in relation to the complete set of unpaginated items.

Request

```
GET https://api.example.org/accounts/?limit=100&offset=400
```

Setup

To enable the `LimitOffsetPagination` style globally, use the following configuration:

```

REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.LimitOffsetPagination'
}

```

Optionally, you may also set a `PAGE_SIZE` key. If the `PAGE_SIZE` parameter is also used then the `limit` query parameter will be optional, and may be omitted by the client.

On `GenericAPIView` subclasses you may also set the `pagination_class` attribute to select `LimitOffsetPagination` on a per-view basis.

Configuration

The `LimitOffsetPagination` class includes a number of attributes that may be overridden to modify the pagination style.

To set these attributes you should override the `LimitOffsetPagination` class, and then enable your custom pagination class as above.

- `default_limit` - A numeric value indicating the limit to use if one is not provided by the client in a query parameter. Defaults to the same value as the `PAGE_SIZE` settings key.
- `limit_query_param` - A string value indicating the name of the "limit" query parameter. Defaults to `'limit'`.
- `offset_query_param` - A string value indicating the name of the "offset" query parameter. Defaults to `'offset'`.
- `max_limit` - If set this is a numeric value indicating the maximum allowable limit that may be requested by the client. Defaults to `None`.
- `template` - The name of a template to use when rendering pagination controls in the browsable API. May be overridden to modify the rendering style, or set to `None` to disable HTML pagination controls completely. Defaults to `"rest_framework/pagination/numbers.html"`.

CursorPagination

The cursor-based pagination presents an opaque "cursor" indicator that the client may use to page through the result set. This pagination style only presents forward and reverse controls, and does not allow the client to navigate to arbitrary positions.

Setup

To enable the `CursorPagination` style globally, use the following configuration, modifying the `PAGE_SIZE` as desired:

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.CursorPagination',
    'PAGE_SIZE': 100
}
```

On `GenericAPIView` subclasses you may also set the `pagination_class` attribute to select `CursorPagination` on a per-view basis.

Configuration

The `CursorPagination` class includes a number of attributes that may be overridden to modify the pagination style.

To set these attributes you should override the `CursorPagination` class, and then enable your custom pagination class as above.

- `page_size` = A numeric value indicating the page size. If set, this overrides the `PAGE_SIZE` setting. Defaults to the same value as the `PAGE_SIZE` settings key.
- `cursor_query_param` = A string value indicating the name of the "cursor" query parameter. Defaults to `'cursor'`.
- `ordering` = This should be a string, or list of strings, indicating the field against which the cursor based pagination will be applied. For example: `ordering = 'slug'`. Defaults to `-created`. This value may also be overridden by using `OrderingFilter` on the view.
- `template` = The name of a template to use when rendering pagination controls in the browsable API. May be overridden to modify the rendering style, or set to `None` to disable HTML pagination controls completely. Defaults to `"rest_framework/pagination/previous_and_next.html"`.